PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

Software Requirements Specification

File Compression Tool (Basic ZIP Implementation)

Version 1.0 approved

Prepared by Chandrakanth V G, Debaditya Chakrabarti, Divisha Gupta, Greeshma N

PES University, Bangalore – Department of Computer Science and Engineering

02-10-2026

PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

Table of Contents

1. Introduction

1.1 Purpose

1.2 Scope

1.3 Definitions, acronyms, and abbreviations

1.4 References

1.5 Overview

2. Overall description

2.1 Product perspective

2.2 Product functions

2.3 User characteristics

2.4 Constraints

2.5 Assumptions and dependencies

3. Specific requirements

3.1 External Interface

3.2 Functional Requirements

The following functional requirements form the complete Version 1.0 baseline. Each requirement is uniquely identified, uses “shall” language, is testable, and is traceable to architecture, design, and verification artifacts.

ID	Feature	Requirement
FR-CMP-01	Input/output handling	The system shall accept a readable input file and a user-specified archive output path.
FR-CMP-02	Frequency analysis	The system shall calculate the frequency of each byte value present in the input file.
FR-CMP-03	Huffman tree construction	The system shall construct a Huffman tree from the calculated byte frequencies.
FR-CMP-04	Bitstream encoding	The system shall encode input bytes into a variable-length Huffman bit stream.

FR-CMP-05	Archive metadata	The system shall store sufficient metadata to reconstruct the encoded data exactly.
FR-CMP-06	Compression reporting	The system shall report original size, compressed size, compression ratio, and operation status.
FR-CMP-07	Lossless round trip	The system shall preserve the exact original byte sequence after compression and decompression.
FR-DEC-01	Format validation	The system shall reject unsupported archive magic identifiers or format versions.
FR-DEC-02	Decoding	The system shall decode a valid archive and reproduce the declared original byte count.
FR-DEC-03	Integrity validation	The system shall reject an archive when checksum/integrity validation fails.
FR-DEC-04	Safe output	The system shall reject unsafe output paths and prevent writing outside the permitted location.
FR-INFO-01	Archive inspection	The system shall display archive metadata without modifying the archive or restored data.
FR-ERR-01	Error reporting	The system shall return a non-zero exit status and actionable message for invalid input or failed operations.
FR-OUT-01	Output safety	The system shall create output only after required validation succeeds and shall not report misleading success.

3.3 Non-Functional Requirements

The non-functional requirements define measurable quality constraints for performance, reliability, usability, maintainability, portability, testability, safety, and security.

ID	Quality Attribute	Requirement
NFR-PERF-01	Performance	On the reference development machine, the tool shall compress and decompress a 10

		MB local reference file within 5 seconds per operation, measured during system testing.
NFR-REL-01	Reliability	For invalid inputs or I/O failures, the tool shall fail without reporting success and shall not leave a trusted partial output.
NFR-USE-01	Usability	For every supported command, the CLI shall provide usage/help information and actionable error text for invalid arguments.
NFR-MAIN-01	Maintainability	Core functionality shall be separated into independently testable modules for controller, Huffman codec, archive handling, file I/O, and validation.
NFR-PORT-01	Portability	The core implementation shall use standard C/C++ facilities and compile with a supported C++17-or-later compiler on the selected development OS.
NFR-TEST-01	Testability	Huffman coding, bit I/O, archive parsing, integrity validation, and controller logic shall expose interfaces suitable for unit and integration testing.
NFR-SAFE-01	Safety	The tool shall not silently overwrite an existing output file and shall remove or invalidate incomplete output after a write failure.
NFR-SEC-01	Security / integrity	The decompressor shall validate archive structure, declared sizes, tree metadata, and checksum/integrity data before accepting decoded output as valid.
NFR-SEC-02	Security / path safety	The decompressor shall reject absolute paths and parent-directory traversal in archive-controlled output names.
NFR-SEC-03	Security / information exposure	The tool shall not log or print full input-file contents;

		diagnostics shall expose only required metadata and errors.
NFR-SEC-04	Security / resource safety	The decompressor shall stop when decoded output exceeds the declared original size or configured safety limit.

3.4 Design Constraints

Appendixes

Index

Revision History

Name	Date	Reason For Changes	Version
Divisha Gupta	02-10-2026	Completed initial project SRS for Huffman-based file compression tool	1.0
Greeshma N	02-10-2026	Reviewed and updated the SRS document	1.0
Team	02-10-2026	Reviewed functional, non-functional and security requirements	1.0

Introduction

Purpose

The File Compression Tool (Basic ZIP Implementation), Version 1.0, is a command-line C/C++ software system that reduces file size using Huffman coding and restores files through lossless decompression. This SRS covers the complete planned software scope: input validation, frequency analysis, Huffman tree construction, canonical/explicit code generation as implemented by the team, bit-level compression, archive creation, decompression, integrity validation, metadata reporting, and error handling.

Intended Audience

The intended audience is the four-member development team, project guide/instructor, software testers, reviewers, and future maintainers. The document is organized according to the supplied PES University SRS template: introduction, overall description, external interfaces, system features and requirements, non-functional requirements, security, quality attributes, business/domain rules, and appendices.

Product Scope

Project description and stakeholder note: This is a self-contained academic software project. No external customer has been identified. Primary stakeholders are the project guide/instructor, student developers/testers, and users who need local lossless file compression. The planned user capabilities are compression, decompression, archive inspection, integrity validation, statistics reporting, and safe error handling.

The product is a self-contained desktop/command-line utility for lossless file compression. It is a simplified ZIP-style implementation whose core compression method is Huffman coding. The project is intended for educational use and is not required to be interoperable with commercial ZIP utilities unless that capability is explicitly added later. The main goals are correct lossless restoration, measurable compression, clear error handling, and a modular C/C++ implementation.

References

1. IEEE Computer Society, IEEE Std 29148-2018, Systems and Software Engineering — Life Cycle Processes — Requirements Engineering.
2. IEEE Std 1016-2009, Systems and Software Engineering — Architecture Description.
3. Project statement: "File Compression Tool (Basic ZIP Implementation) – A basic ZIP compression tool using Huffman coding for file size reduction."
4. C/C++ standard library documentation for binary file I/O and command-line processing.
5. Huffman, D. A., "A Method for the Construction of Minimum-Redundancy Codes," Proceedings of the IRE, 1952.

Overall Description

Product Perspective

The tool is a new, self-contained software project rather than a replacement for an existing product. The user interacts through a command-line interface. During compression, the system reads the input file as bytes, computes symbol frequencies, builds a Huffman tree, generates codes, writes archive metadata and the encoded bit stream, and records integrity information. During decompression, the system validates the archive, reconstructs the Huffman tree, decodes the bit stream, and recreates the original file.

Product Functions

- *Compress a selected input file using Huffman coding.*
- *Decompress a generated archive and restore the original file losslessly.*
- *Display compression statistics such as original size, compressed size, and compression ratio.*
- *Store and read sufficient archive metadata to reconstruct the Huffman coding tree.*

• *Validate archive structure and integrity before decompression.*
• *Reject unsupported, missing, corrupted, or malformed input with clear errors.*
• *Prevent accidental overwrite of an existing output file unless explicitly permitted.*
• *Support deterministic command-line operations for compression, decompression, and metadata inspection.*

User Classes and Characteristics

Primary User – Student/developer or general desktop user who provides a file and invokes compression/decompression commands. The user is expected to understand basic command-line usage but does not need knowledge of Huffman coding.

Developer/Maintainer – Team member who understands C/C++ and the internal modules and is responsible for implementation, debugging, and maintenance.

Tester/Reviewer – Project evaluator who executes defined test cases and verifies functional, security, and quality requirements.

Operating Environment

The software is intended to run on a modern desktop/laptop environment supporting a C/C++ compiler. Development and testing may be performed on Windows, Linux, or macOS with a standard C++17-or-later toolchain. The tool uses local file-system access and standard console input/output; no network connection is required.

Design and Implementation Constraints

• Implementation language: C or C++ as permitted by the project statement; the design assumes C++17 for the submitted implementation.
• Compression must be lossless; decompression must reproduce the original byte sequence.
• Huffman coding is the required compression technique.
• The archive format is a simplified project-defined ZIP-style format and is not assumed to be compatible with commercial ZIP tools.
• Only local file-system operations are required; cloud storage and network transfer are out of scope.
• The implementation must use binary-safe file I/O and handle large files without loading the entire compressed output into memory where avoidable.
• No third-party compression library may replace the required Huffman implementation.

2.6 Assumptions and Dependencies

• The operating system provides permission to read the source file and create the output archive/restored file.
• A standards-compliant C/C++ compiler and standard library are available.
• Input files remain unchanged while compression is running.
• The project team will define and maintain the archive header format consistently across compression and decompression modules.
• Dependencies include the C/C++ standard library and the local file system; no external service is required.

External Interface Requirements

User Interfaces

The command-line interface shall accept explicit operations and file paths. Example forms are: `compress <input> <archive>`, `decompress <archive> <output>`, and `info <archive>`. The interface shall print concise success/failure messages, compression statistics, and actionable error messages. Passwords or sensitive user data are not collected.

Software Interfaces

The system shall use the C/C++ standard library for file streams, string handling, containers, and command-line processing. Internal software interfaces shall separate the command controller, Huffman codec, archive manager, file I/O layer, and integrity validator. The archive manager shall exchange a defined header, metadata, and compressed payload with the file I/O layer.

Communications Interfaces

No network communication is required. All communication between modules occurs through function calls and in-memory data structures. File transfer uses local binary I/O. If the project is later extended with a GUI or network service, the communication protocol shall be specified separately.

Hardware Interfaces: None required. The project is a software-only command-line utility and does not directly control dedicated hardware.

Analysis Models

The primary analysis model is the UML use case diagram included below. The diagram identifies the User/Operator and the major system services: compress file, decompress archive, view archive metadata, validate archive integrity, and error handling. These use cases map directly to the functional requirements and test cases in this document.

Figure 1. UML Use Case Diagram

System Features

This section organizes functional requirements around the two major user services: compression and decompression/validation. Each requirement is uniquely identified and written to be clear, concise, measurable where applicable, and verifiable.

Compress File

Compress File

5.1.1 Description and Priority

High priority. This feature performs lossless Huffman compression and produces a project-defined archive containing the metadata required for restoration.

5.1.2 Stimulus/Response Sequences

1. User supplies a valid input path and output archive path.
2. System verifies that the input exists and is readable.
3. System scans the file and calculates byte frequencies.
4. System builds the Huffman tree and generates codes.
5. System writes archive header, metadata, and compressed bit stream.
6. System validates the written archive and reports output size and compression ratio.
7. If any input/output error occurs, the system stops safely and reports the error without producing a misleading success result.

5.1.3 Functional Requirements

FR-CMP-01: The system shall accept a readable input file and a user-specified output archive path.
FR-CMP-02: The system shall calculate the frequency of each byte value present in the input file.
FR-CMP-03: The system shall construct a Huffman tree from the calculated frequencies.
FR-CMP-04: The system shall encode the input bytes into a variable-length Huffman bit stream.
FR-CMP-05: The system shall store archive metadata sufficient for exact decompression.
FR-CMP-06: The system shall report original size, compressed size, and compression ratio after successful compression.
FR-CMP-07: The system shall preserve the exact original byte sequence after a compress/decompress cycle.

Each requirement is uniquely identified using a stable requirement ID so that it can be traced to architecture, design and test artifacts.

FR-CMP-01: The system shall reject a missing or unreadable input file and display an error message within 2 seconds of command invocation.
FR-CMP-02: The system shall create an archive only when the complete compression operation finishes successfully; partial output shall be removed or marked invalid on failure.

Decompress Archive

High priority. This feature validates a project-generated archive, reconstructs the Huffman representation, decodes the bit stream, and restores the original file.

5.2.1 Description and Priority

High priority. The decompression operation is required to restore the original byte sequence from a valid project archive.

5.2.2 Stimulus/Response Sequences

1. User supplies an archive path and output path.
2. System verifies archive existence and readability.
3. System validates magic/version, metadata, declared size, and integrity information.
4. System reconstructs the Huffman tree and decodes the bit stream.
5. System writes the restored file only after safe validation checks.
6. System reports success and restored size, or returns a specific error with a non-zero exit status.

5.2.3 Functional Requirements

FR-DEC-01: The system shall reject an archive with an unsupported magic identifier or format version.
FR-DEC-02: The system shall decode a valid archive and reproduce exactly the declared original byte count.
FR-DEC-03: The system shall reject an archive when checksum/integrity validation fails.

FR-DEC-04: The system shall reject unsafe output paths and shall not write outside the permitted output location.

Other Nonfunctional Requirements

Performance Requirements

NFR-PERF-01: For a 10 MB local text/binary test file on the reference development machine, the system should complete compression or decompression within 5 seconds, excluding OS-level file-copy delays. Performance shall be measured during system testing.

Safety Requirements

SAF-01: The system shall never silently overwrite an existing output file. It shall require an explicit overwrite option or choose a non-conflicting output path.
SAF-02: On write failure, the system shall not report success and shall attempt to remove the incomplete output file.

Security Requirements

SEC-OBJ-01: Protect archive integrity by detecting malformed or modified archive metadata/payload before producing a trusted decompressed output.
SEC-OBJ-02: Prevent unsafe file-system behavior such as path traversal through archive metadata and accidental overwrite of existing files.
SEC-REQ-01: The system shall validate archive magic/version, declared sizes, tree metadata, and checksum/integrity information before or during decompression and shall reject invalid archives.
SEC-REQ-02: The system shall sanitize the restored output filename/path and shall reject absolute paths, parent-directory traversal (`..`), and invalid path components supplied through archive metadata.
SEC-REQ-03: The system shall not log or display full file contents; diagnostic output shall contain only required metadata and error details.
SEC-REQ-04: The decompressor shall enforce declared output-size limits and shall stop if decoding would exceed the declared original size.

Security Objective SO-01: Preserve archive integrity and detect malformed or modified metadata/payload before trusted restoration.
Security Objective SO-02: Prevent archive-controlled file-system operations from causing path traversal, unsafe writes, or accidental overwrite.
Security Requirement SEC-REQ-01: Validate magic/version, declared sizes, tree metadata, and checksum/integrity information and reject invalid archives.
Security Requirement SEC-REQ-02: Reject absolute output paths, parent-directory traversal, and invalid path components from archive metadata.
Security Requirement SEC-REQ-03: Do not log or display complete file contents.
Security Requirement SEC-REQ-04: Enforce declared output-size limits and stop decoding if the limit would be exceeded.

Security requirements are verified through malformed-archive, tampering, path-traversal, and output-size abuse tests.

Software Quality Attributes

Correctness: decompression must reproduce the original byte sequence.
Reliability: failures shall be detected and reported without false success.
Performance: the implementation should meet the stated 10 MB/5 second reference target.
Maintainability: modules shall have clear interfaces and single responsibilities.
Portability: the core implementation shall rely on standard C/C++ facilities.
Usability: command syntax and error messages shall be concise and understandable.
Testability: compression, Huffman coding, archive parsing, and validation shall be independently testable.

Business Rules

BR-01: Compression shall be lossless; the original file must be recoverable exactly.
BR-02: An archive must contain enough metadata to reconstruct the same coding tree or equivalent decoding representation.
BR-03: Decompression shall process only archives produced by the supported project format/version.
BR-04: Invalid input shall never be treated as a successful operation.
BR-05: Output files shall not be overwritten by default.

Domain requirement: Huffman coding assigns shorter codes to more frequent symbols and longer codes to less frequent symbols. The encoded stream must therefore be decodable using the stored tree/code representation. The implementation shall support the complete byte range 0–255.

Other Requirements

Archive metadata requirements: magic identifier, format version, original file size, compressed payload size or equivalent end marker, tree/code representation, original filename metadata, and integrity/checksum field. The project shall not depend on a database. The archive format shall be documented in the design specification.

Appendix A: Glossary

Byte: 8-bit unit of input data.
Huffman tree: Binary tree used to derive prefix-free variable-length codes.
Huffman code: Bit sequence assigned to a byte/symbol.
Archive: Project-defined file containing metadata and compressed payload.
Bit stream: Sequence of individual encoded bits stored in bytes.

Compression ratio: Compressed size divided by original size, expressed as a percentage where appropriate.
Checksum: Value used to detect accidental or unauthorized changes to data.
Lossless compression: Compression in which decompression reproduces the original data exactly.

Appendix B: Field Layouts

Archive Metadata Field Layout – the following table replaces the banking sample in the supplied template and describes the fields relevant to this project.

Sample Archive Metadata Layout

Field	Length	Data Type	Description	Is Mandatory
Magic Number	4	Byte[4]	Identifies supported project archive format	Y
Version	2	Archive format version		Y
Original File Name	255	String	Sanitized source file name	Y
Original Size	8	Unsigned Integer	Number of original bytes	Y
Compressed Size	8	Unsigned Integer	Number of compressed payload bytes	Y
Symbol Count	2	Unsigned Integer	Number of stored symbols/tree entries	Y
Tree Metadata	Variable	Binary	Huffman tree/code representation	Y
Checksum	4	Unsigned Integer	Integrity value for original data	Y

Sample Report Requirements: Include the fields to be included in the report

Compression Report	**Validation / Decompression Report**
Input File Name	Archive File Name
Original Size	Declared Original Size
Compressed Size	Decoded Output Size
Compression Ratio	Checksum / Integrity Status
Huffman Symbol Count	Archive Version
Output Archive	Restored Output File
Execution Time	Validation Result
Status	Error / Warning Details
SI Start Date	Transaction Date
Status	Status
Remarks	Debit Attempt Number
	Remarks

Appendix C: Requirement Traceability Matrix

Sl. No	Requirement ID	Brief Description of Requirement	Architecture Reference	Design Reference	Code File Reference	Test Case ID	System Test Case ID
1	FR-CMP-01	Accept valid input/output paths	Compression Controller + File I/O	3.1 CLI / 4.3 API	main.cpp, file_io.cpp	TC-01	ST-01
2	FR-CMP-02	Calculate byte frequencies	Huffman Codec	4.1 Frequency analysis	huffman.cpp	TC-02	ST-02
3	FR-CMP-03	Construct Huffman tree	Huffman Codec	4.2 Tree construction	huffman.cpp	TC-03	ST-03
4	FR-CMP-04	Encode bytes into bit stream	Huffman Codec	4.3 Encoding	bit_io.cpp	TC-04	ST-04
5	FR-CMP-05	Store metadata for decompression	Archive Manager	4.4 Archive format	archive.cpp	TC-05	ST-05
6	FR-CMP-06	Report compression statistics	Compression Controller	4.5 Reporting	main.cpp	TC-06	ST-06
7	FR-CMP-07	Restore exact original bytes	Huffman Codec + Archive Manager	4.6 Decompression	huffman.cpp, archive.cpp	TC-07	ST-07
8	SEC-REQ-01	Reject malformed/tampered archives	Integrity Validator	4.7 Validation	validator.cpp	TC-08	ST-08
9	SEC-REQ-02	Prevent unsafe output paths	Archive Manager	4.8 Path validation	archive.cpp	TC-09	ST-09
10	NFR-PERF-01	Meet reference performance target	File I/O + Huffman Codec	4.9 Performance	benchmark.cpp	TC-10	ST-10
11	FR-DEC-01	Reject unsupported archive format	Archive Manager + Validator	Decompression validation	archive.cpp, validator.cpp	TC-08	ST-08
12	FR-DEC-02	Restore exact declared byte count	Huffman Codec	Decode flow	huffman.cpp	TC-07	ST-07
13	FR-DEC-03	Reject checksum/integrity failure	Integrity Validator	Integrity validation	validator.cpp	TC-08	ST-08
14	FR-DEC-04	Reject unsafe output paths	Archive Manager	Path validation	archive.cpp	TC-09	ST-09
15	FR-INFO-01	Inspect archive metadata	Archive Manager	Info command / metadata parser	archive.cpp, main.cpp	TC-05	ST-05
16	FR-ERR-01	Actionable errors and non-zero status	Controller + File I/O	Error handling design	main.cpp, errors.cpp	TC-08	ST-08

17	FR-OUT-01	Safe output creation	Archive Manager + File I/O	Validated output flow	archive.cpp, file_io.cpp	TC-09	ST-09
18	NFR-REL-01	Fail without false success	Controller + Validator	Failure-state handling	main.cpp, validator.cpp	TC-08	ST-08
19	NFR-USE-01	Usable CLI/help/error messages	CLI Controller	Command /help interface	main.cpp	TC-01	ST-01
20	NFR-MAIN-01	Modular maintainable design	All components	Layered module interfaces	*.cpp / *.h	Review-01	ST-11
21	NFR-PORT-01	C++17 portability	All components	Standard library interfaces	*.cpp / *.h	Build-01	ST-12
22	NFR-TEST-01	Independent testability	All testable modules	Module APIs	*.cpp / *.h	Unit-01	ST-13
23	NFR-SAFE-01	No silent overwrite / cleanup	File I/O + Archive Manager	Output safety controls	file_io.cpp, archive.cpp	TC-09	ST-09
24	NFR-SEC-01	Archive integrity validation	Integrity Validator	Validation pipeline	validator.cpp	TC-08	ST-08
25	NFR-SEC-02	Path traversal protection	Archive Manager	Path validation	archive.cpp	TC-09	ST-09
26	NFR-SEC-03	No full-content logging	Controller / Logger	Diagnostic policy	errors.cpp, main.cpp	TC-08	ST-08
27	NFR-SEC-04	Output-size/resource limit	Integrity Validator + Decoder	Declared-size guard	validator.cpp , huffman.cpp	TC-08	ST-08

Appendix D: Team Profile and Plan of Work

Team / Group: Group 14 – File Compression Tool (Basic ZIP Implementation)

Team Profile

1. PES1UG25CS810 – Chandrakanth V G

2. PES1UG24CS140 – Debaditya Chakrabarti

3. PES1UG24CS158 – Divisha Gupta

4. PES1UG24CS175 – Greeshma N

Short-Term Plan of Work

Week 1: finalize requirements, archive format, CLI commands, module interfaces, and UML models.

Week 2: implement frequency analysis, Huffman tree/code generation, bit-level encoding/decoding, and binary file I/O.

Week 3: implement archive creation/parsing, metadata, integrity validation, error handling, and command-line integration.

Week 4: integrate all modules, execute functional/security/performance tests, fix defects, document results, and prepare final demonstration.

Product Ownership / Allocation

Chandrakanth V G – Integrity validation, error handling, security validation, and testing: checksum/integrity checks, malformed archive handling, path validation, performance tests, and defect tracking; quality target: reliable failure handling and security test coverage.

Debaditya Chakrabarti – Archive format and file I/O: archive header/metadata serialization, binary read/write, output handling, and metadata inspection; quality target: robust parsing and portability of file operations.

Divisha Gupta – Developer / Algorithm Lead: Huffman algorithm module, including frequency table, priority queue/tree construction, code generation, and encode/decode functionality; quality target: correctness of tree/code generation and unit-test coverage.

Greeshma N – Architecture, controller/CLI integration, requirements traceability, and overall integration: command processing, module interfaces, documentation, and integration testing; quality target: maintainable modular design and complete requirement traceability.

All members shall participate in requirements analysis, design review, code review, testing, debugging, documentation, and project presentation.